

Clasificación de imágenes en tiempo real usando MobileNet y Keras

Objetivo:

Capturar una imagen desde la webcam y clasificar su contenido usando un modelo CNN preentrenado de Keras (MobileNet).

Paso 0: Instalar bibliotecas necesarias

```
pip install tensorflow opencv-python
```

Se asume que estás usando Python 3.8 o superior.

Paso 1: Importar las librerías

```
import cv2
```

```
import numpy as np
```

```
from tensorflow.keras.applications.mobilenet import MobileNet, decode_predictions, preprocess_input
```

```
from tensorflow.keras.preprocessing.image import img_to_array
```

Explicación:

- MobileNet: modelo ligero para clasificación de imágenes.
- decode_predictions: convierte la salida del modelo en etiquetas legibles.
- preprocess_input: prepara la imagen en el formato que MobileNet espera.
- cv2: para capturar imágenes.
- img_to_array: convierte la imagen en un array compatible.

Paso 2: Capturar una imagen desde la webcam

```
cap = cv2.VideoCapture(0)
```

```
print("Presiona ESPACIO para capturar una imagen")
```

```
while True:
```

```
    ret, frame = cap.read()
```

```
    cv2.imshow("Webcam - Presiona ESPACIO para capturar", frame)
```

```
    key = cv2.waitKey(1)
```

```
    if key == 27: # ESC para salir
```

```
        cap.release()
```

```
        cv2.destroyAllWindows()
```

```
        exit()

    elif key == 32: # ESPACIO

        image = frame

        break

cap.release()

cv2.destroyAllWindows()
```

Explicación:

- Abre la cámara.
- Muestra el video en vivo.
- Captura la imagen cuando se presiona **ESPACIO (key 32)**.

Paso 3: Preprocesar la imagen

```
resized_image = cv2.resize(image, (224, 224)) # Tamaño esperado por MobileNet

image_array = img_to_array(resized_image)

image_array = np.expand_dims(image_array, axis=0)

image_array = preprocess_input(image_array)
```

Explicación:

- Redimensiona a 224x224 píxeles.
- Convierte a array, añade una dimensión extra (batch) y normaliza los datos.

Paso 4: Cargar el modelo MobileNet y hacer la predicción

```
model = MobileNet(weights='imagenet')

predictions = model.predict(image_array)
```

Explicación:

- `MobileNet(weights='imagenet')`: carga el modelo ya entrenado con el dataset de **ImageNet** (más de 1000 clases).
- `predict()`: obtiene la predicción del modelo.

Paso 5: Mostrar la predicción

```
decoded = decode_predictions(predictions, top=3)[0]

for i, (imagenetID, label, prob) in enumerate(decoded):
```

```
print(f'{i+1}. {label}: {prob*100:.2f}%')
```

Explicación:

- `decode_predictions()` convierte los resultados en etiquetas comprensibles como “banana”, “remote control”, etc.
- Se muestran las **3 predicciones más probables** con su nivel de confianza.

Código completo:

```
import cv2

import numpy as np

from tensorflow.keras.applications.mobilenet import MobileNet, decode_predictions,
preprocess_input

from tensorflow.keras.preprocessing.image import img_to_array


# Capturar imagen

cap = cv2.VideoCapture(0)

print("Presiona ESPACIO para capturar una imagen")


while True:

    ret, frame = cap.read()

    cv2.imshow("Webcam - Presiona ESPACIO para capturar", frame)

    key = cv2.waitKey(1)

    if key == 27: # ESC

        cap.release()

        cv2.destroyAllWindows()

        exit()

    elif key == 32: # ESPACIO

        image = frame

        break


cap.release()

cv2.destroyAllWindows()
```

```

# Preprocesamiento

resized_image = cv2.resize(image, (224, 224))

image_array = img_to_array(resized_image)

image_array = np.expand_dims(image_array, axis=0)

image_array = preprocess_input(image_array)


# Modelo y predicción

model = MobileNet(weights='imagenet')

predictions = model.predict(image_array)

decoded = decode_predictions(predictions, top=3)[0]


# Mostrar resultados

for i, (imagenetID, label, prob) in enumerate(decoded):

    print(f"{i+1}. {label}: {prob*100:.2f}%")

```

Actividades:

1. **Clasifica objetos del aula:**
 - Intenta clasificar una mochila, un lápiz, una botella, etc.
 - ¿Aparece el nombre correcto? ¿Qué porcentaje da? Adjunta evidencias
2. **Actividad adicional:**
 - Captura diferentes objetos y registra los resultados en una tabla agrega la evidencia:

Objeto	Predicción Top 1	Confianza (%)
Botella de agua	"water bottle"	91.5%
Mouse	"computer mouse"	84.3%

3. **Reflexión final:**
 - ¿Qué ventajas tiene este modelo frente a Haar?
 - ¿Qué limitaciones observaron?